Internally, a Ruby object has three components: a set of flags, some instance variables, and an associated class. A Ruby class contains all the things an object has plus a set of method definitions and a reference to a superclass (which is itself another class). (A Ruby class is itself an instance of the class Class.) Let's look at how that structure lends itself to metaprogramming in Ruby.

Method Calling and "self"

Ruby has a concept of the *current object*. This current object is referenced by the built-in, read-only variable self. The first time that Noel heard Dave Thomas speak in public, the topic was Ruby metaprogramming and he said "understanding *self* is the key to Ruby. Also the key to life". Words to live by.

The value self has two significant roles in a running Ruby program. First, self controls how Ruby finds instance variables. We already said that every object carries around a set of instance variables. When you access an instance variable with the @ syntax, Ruby looks for that variable in the object that is referenced by self in the context that instance variable has been accessed.

Second, self plays a vital role in method calling. In Ruby, each method call is a message passed to some object. This object is called the *receiver* of the call. When you make a call such as items.size, the object on the left side of the dot—here referenced by the variable items—is the receiver and size is the method to invoke.

Often, you will see a method call where there is no explicit receiver, such as puts "hi". In this case, Ruby uses the current object, self, as the receiver. It goes to self's class and looks up the method (in this case, puts). If it can't find the method in the class, it looks in the class's superclass and then in that class's superclass, stopping when it runs out of superclasses (which will happen after it has looked in BasicObject). If it can't find the method in the object's class hierarchy, Ruby looks for a method called method_missing on the original receiver, starting back at the class of self and then looking up the superclass chain. (In the case of puts, Ruby will find the method defined in the Kernel module that is included in Object.)

When you make a method call with an explicit receiver (for example, invoking items.size), the process is similar. The only change—but it's a vitally important one—is that the value of self is changed for the duration of the call. Before starting the method lookup process, Ruby sets self to the explicit receiver (the object referenced by items in this case). Then, after the call returns, Ruby restores self to the value it had before the call.

Let's see how this works in practice. Here's a simple program:

```
class Test
  def one
    @var = 99
    two
  end

  def two
    puts @var
  end
end

t = Test.new
t.one
```